PART V — TOKENS, CONTEXT, AND MODEL CHOICE

Chapter 27

Agent State Engineering Beyond the System Prompt

"The most reliable instruction in an agentic loop is the one the model never has to remember, because the system never let it forget in the first place."

Chapter 27 — Agent State Engineering Beyond the System Prompt

The answer this project's own `agent_query.py` already gives, in code written well before this book named the pattern, is to stop relying on the system prompt as the sole place state lives. Retrieval counts, tried queries, and retry context are tracked in Python variables and re-injected into the conversation at exactly the moments the model needs them — not trusted to the model's own memory of a system prompt read many tokens ago.

27.1 Why "remind the model from the system prompt" stops working at scale

A tempting first response to Chapter 25's failures is to add more reminders to the system prompt — restate the iteration count, restate which queries are already tried, restate the retrieval cap, every single turn. Chapter 23.4 already quantifies why this does not scale: every reminder is itself tokens, competing for the same limited instruction-following budget Chapter 24.3 measured at roughly 1,800 tokens for the 8B model. A system prompt that grows to hold a full, ever-lengthening restatement of "here is everything that has happened so far" eventually becomes the exact bloated, diluted prompt ADR-012 was written to fix.

The alternative this project's own code demonstrates is not "remind harder" but "remind precisely, at the moment of use, in the channel most likely to be read." Rather than one growing system-prompt block trying to cover every possible future need, state is injected in small, targeted pieces exactly where and when a specific decision depends on it — which is Section 27.2 through 27.4's actual subject.

This distinction is worth stating as a general design rule, not just a description of one codebase's choices: a system prompt is written once per session and read on every subsequent call, which makes it the wrong place for anything that changes turn to turn. State that changes — how many retrievals have run, which queries already failed, what the last verdict said — belongs in a location that updates itself automatically as the loop progresses, not in a block of static text a developer would otherwise need to manually rewrite on every iteration just to keep it current.

27.1B What counts as state versus what counts as instruction

It helps to draw a sharp line between two things this chapter's mechanisms both touch: instructions (how the model should behave, established once in Chapter 26's prompt structure) and state (facts about what has already happened, which change every turn). `_ROLE_AND_RULES` and `_PROCESS_INSTRUCTIONS` are instructions — they do not change within a session. `total_retrievals`, `all_tried_queries`, and the current iteration count are state — they change on nearly every turn of the loop. Conflating the two, by trying to encode changing facts as if they were static instructions, is precisely what produces the token-bloat problem Section 27.1 already diagnosed.

27.2 Injecting retrieval state into every tool result

`_handle_retrieve_documents_call()` in `agent_query.py` does not merely return retrieved chunks — when a limit is reached, it returns state directly as the tool's own result text: `"Retrieval limit reached. Use what you have to answer."` when `total_retrievals >= MAX_TOTAL_RETRIEVALS`. This is state injection at the single most recency-favorable position available: a tool result is, by construction, among the most recent tokens the model reads before its next decision — Chapter 26.1's recency-bias placement, achieved automatically, for free, as a side effect of how tool calls work.

```
if total_retrievals >= MAX_TOTAL_RETRIEVALS:
    result = "Retrieval limit reached. Use what you have to answer."
    logger.warning(f"    [SKIPPED] retrieval cap ({MAX_TOTAL_RETRIEVALS})
reached")
    return result
```

The same pattern appears in the retrieval judge's sidecar message, appended as its own `{"role": "tool", ...}` entry whenever a retrieval verdict is FAIL or PARTIAL — `[RETRIEVAL JUDGE] The retrieval tracks were validated separately...` — a second, independent channel carrying state (this batch's relevance quality) into the conversation at the moment it is most actionable, rather than folded into a system-prompt paragraph the model would need to actively recall several turns later.

Both messages share a structural property worth naming: they are self-terminating. `"Retrieval limit reached. Use what you have to answer."` does not merely inform — it closes off a path, functioning simultaneously as Section 27.4's hard filter and as the state signal explaining why that filter fired. A model reading this text has both the fact (the cap was hit) and the consequence (stop retrieving, proceed to answer) delivered in the same nine words, at the exact turn where both pieces of information are needed and nowhere else.

27.3 The per-iteration state-summary message

When the JUDGE phase returns INSUFFICIENT with retrieval budget still remaining, `run_agent()` does not simply loop back to RETRIEVE silently — it appends a fresh `{"role": "user", ...}` message summarizing exactly what happened and what to do differently, built fresh on every re-loop rather than accumulated as one growing block.

```
retry_msg = (
    f"Your previous answer was judged INSUFFICIENT by the quality
checker.\n"
    f"Reason: {reason}\n\n"
    f"Please call retrieve_documents again with 1-2 NEW, SEMANTICALLY
DIFFERENT "
    f"query variants that approach the topic from a fresh angle. "
    f"Do NOT repeat any query you have already tried."
)
messages.append({"role": "user", "content": retry_msg})
```

This message plays the identical role a status update plays in a long conversation between two people who keep losing the thread: it does not ask the model to recall the whole history, it hands the model a compact, current summary of exactly the two facts it needs right now — why the last attempt failed, and what to try differently — placed at the exact point recency bias weights most heavily, immediately before the decision it should inform.

Analogy — The relay runner's baton, not the full race recap

A relay runner does not need to hear a replay of every prior leg of the race before taking the baton — only the current pace and what's left to run. `retry_msg` is the baton handoff: the reason for the last failure and the next concrete instruction, nothing more, passed at exactly the moment it matters.

27.4 Hard filters at the tool layer

Definition — Hard filter

A check performed in code, before a request ever reaches the model's decision-making, that refuses or short-circuits an action outright — as distinct from a soft instruction that merely asks the model not to take that action.

The retrieval-cap message from Section 27.2 and the duplicate-query message below it in the same function are not merely informational text — they are the visible surface of a hard filter that has already made its decision before the model's prior instruction-following even comes into play. `all_tried_queries` is a Python set, checked with a normalized string comparison, before the model's chosen query is ever sent to the retriever.

```
q_normalized = args["query"].strip().lower()
if q_normalized in all_tried_queries:
    return "You already tried this exact query. Try a genuinely different
angle."
all_tried_queries.add(q_normalized)
```

This is Chapter 25.3's soft-versus-hard distinction applied at the smallest possible granularity: `_PROCESS_INSTRUCTIONS`'s "Never repeat a query already tried" is the soft version of this exact rule, stated in the system prompt as a bias on the model's behavior. The `all_tried_queries` check is the hard version — the query is rejected regardless of whether the model's instruction-following held on this particular turn. Reliability here does not depend on the prompt working; the prompt and the filter work together, with the filter as the guarantee and the prompt as the reason the filter rarely has to intervene at all.

The `MAX_TOTAL_RETRIEVALS` check from Section 27.2 is the identical pattern at the level of a whole session rather than a single query: `_ROLE_AND_RULES` states the soft version — "retrieve_documents: max 5 calls total" — and the code-level comparison `if total_retrievals >= MAX_TOTAL_RETRIEVALS` is the hard version underneath it. Both this and the query-deduplication filter share the same shape: a stated rule in the prompt, and an unconditional check in code that holds even on the turn the model's instruction-following happens to fail.

It is worth being precise about what a hard filter actually buys, because it is not "the model never tries the forbidden action." The model can and sometimes will emit a duplicate query or attempt a sixth retrieval call — Chapter 25's soft instruction-following limits do not disappear just because a filter exists downstream. What the filter buys is that the *action never takes effect*: the duplicate query never reaches the retriever, the sixth retrieval attempt never consumes a real API call. The model's mistake becomes harmless, caught and neutralized in code, rather than something the prompt has to somehow prevent from happening in the first place.

27.5 Compressing tool results before appending to history

`compress_context()`'s chat-history scrubbing — replacing every raw `retrieve_documents` tool-result message with `COMPRESSED_PLACEHOLDER` once compression runs — is itself a state-engineering move, not merely a token-budget one. Leaving stale raw retrieval results in history after they have already been superseded by a compressed, consolidated context is a second source of truth the model could accidentally read from instead of the canonical compressed version — precisely the kind of drift Section 27.6 argues against.

The token-cost side of this replacement is real and worth estimating concretely, even without a project-recorded percentage figure to cite directly. ADR-015 fixes chunk size at roughly 250 tokens; a single raw `retrieve_documents` result carrying five chunks costs on the order of 1,250 tokens (Chapter 23.4's own calculation), while `COMPRESSED_PLACEHOLDER` is a fixed, short sentence of well under 50 tokens. Replacing even one such message is a reduction on the order of 95% for that specific message alone — the aggregate savings across a multi-retrieval session scales with however many raw results compression is able to retire from history at once.

A session with the full 2-3 pre-compression retrieval calls `_PROCESS_INSTRUCTIONS` permits, each carrying roughly 1,250 tokens of raw chunk text, holds 2,500-3,750 tokens of now-superseded content in history right up until `compress_context` runs. Scrubbing that content is not optional bookkeeping — left in place, it is exactly the kind of middle-of-prompt bulk Chapter 24.1's lost-in-the-middle effect predicts the model will attend to unreliably, made worse by the fact that this particular bulk is not merely low-value but actively stale, describing chunks the compressed context has already superseded.

27.6 The single-source-of-truth principle

Every mechanism in this chapter reads from or writes to the same place: `agent_state`, `total_retrievals`, `all_tried_queries`, and `iterations` are each tracked exactly once, in code, and every consumer — the tool-result filter, the retry message, the final answer's iteration-count log line — reads that single value rather than maintaining its own separate count. Figure 27.2 makes the contrast explicit: one counter with three readers cannot disagree with itself; three independent ad-hoc counters, one per consumer, can silently drift the moment any single one of them is updated without updating the others.

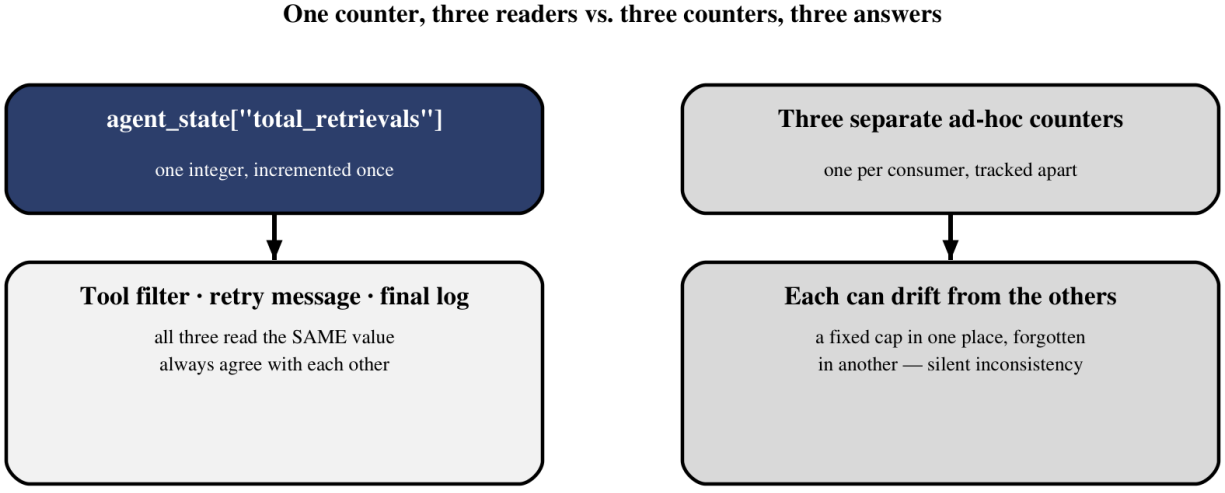

Figure 27.2 — A single counter shared by every consumer cannot disagree with itself; separately tracked counters can silently drift apart.

Figure 27.1 places all four mechanisms from this chapter on one spectrum, from the softest (a system-prompt sentence, Chapter 26) to the hardest (a code-level filter that never even surfaces as text, Section 27.4), with `agent_state` as the shared foundation every layer above it ultimately reads from or writes to.

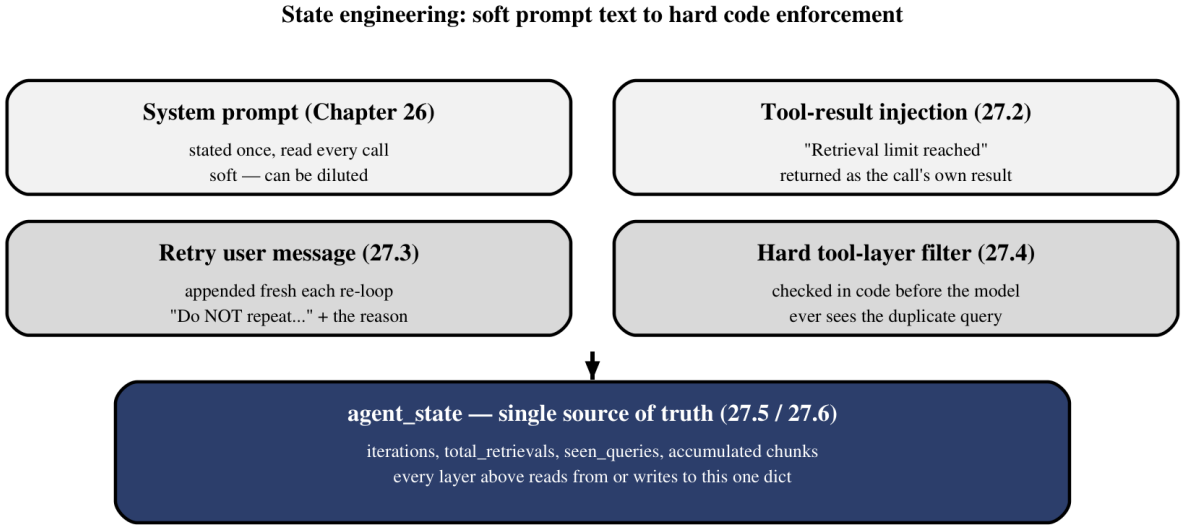

Figure 27.1 — Four state-engineering layers, ordered from softest to hardest, all grounded in the same underlying agent_state.

The principle generalizes well beyond this specific pipeline: any fact a system depends on for correctness — a count, a flag, a set of things already tried — should have exactly one authoritative home. The system prompt, tool results, and retry messages are all legitimate *channels* for surfacing that fact to the model at the right moment, per Chapter 26's recency discipline, but none of them should be where the fact actually *lives*. That distinction — one source of truth, many channels for communicating it — is the thread running through every guardrail this project's own agentic loop relies on, and it closes the argument Part V opened with Chapter 23's token budget: a small model operating inside a finite window stays reliable not because it remembers everything correctly, but because the system around it was engineered to need it to remember as little as possible.

Every mechanism this chapter named — hard filters, injected state, a single authoritative counter — governs state that lives and dies with one query's `run_agent()` call. Part VI turns to a different, longer-lived kind of state: what a system remembers across sessions, once a query is finished and the next user arrives asking something the system may already, in some sense, have learned.

That shift matters because none of the within-session guarantees this chapter built — the single counter, the hard filter, the self-terminating tool result — automatically extend across a session boundary. A `total_retrievals` count reset to zero at the start of the next query is correct behavior for that variable's actual scope. Part VI's subject is a different question entirely: not how a single run stays internally consistent, but what, if anything, should persist once that run ends and be available the next time a related question arrives.